Grafos dirigidos y algoritmos de caminos mínimos

Informe

Juan David Romero Morales – 20222020102

Jaider Alejandro González Rojas - 20242020075

Jimmy Alejandro Granados Becerra - 20251020034

Juan Sebastian Gutiérrez Poveda - 20251020199

Universidad Distrital Francisco Jose de Caldas

Ciencias de la Computación II – Grupo 020-84

Docente: Simar Enrique Herrera

Bogotá D.C.

19 De Septiembre De 2026

Tabla de Contenido

Pág.

1. Introducción 3

2. Objetivos 4

2.1. Objetivo General 4

2.2. Objetivos Específicos 4

3. Marco Teórico y Conceptos Clave 5

3.1. Grafos dirigidos y ponderados 5

3.2. Caminos y distancias mínimas 5

3.3. Algoritmos implementados 5

3.4. Conceptos utilizados 5

4. Explicación del Código Desarrollado 7

4.1. Funcionamiento General 7

4.2. Detalle Profundo de las Funciones 7

4.3. Flujo de Ejecución del Sistema 10

5. Pruebas 11

6. Conclusión 16

1. Introducción

LINK DEL REPOSITORIO: https://github.com/Jimmy86gb/CC2_GR5_020_84.git

Ruta de Entrega: 2DO_CORTE/2_Algoritmos_Caminos_Mas_Cortos/Entrega

Los grafos son una de las estructuras de datos más usadas para modelar problemas reales: redes de carreteras, rutas de transporte, redes de comunicación o dependencias entre tareas. En muchos de estos problemas interesa conocer el camino de menor costo entre dos puntos, lo que se conoce como el problema del camino mínimo.

Este proyecto implementa en C++ un grafo dirigido y ponderado representado mediante listas de adyacencia, junto con tres algoritmos clásicos de caminos mínimos: Dijkstra, Bellman–Ford y Floyd–Warshall. El programa ofrece un menú interactivo que permite construir el grafo (agregar y eliminar nodos y aristas), visualizar su estructura y ejecutar cada algoritmo. Además, incluye dos grafos de prueba precargados, uno con pesos positivos y otro con un peso negativo.

El informe describe los fundamentos teóricos, explica el funcionamiento del código y presenta las pruebas realizadas. Estas pruebas comparan el comportamiento de los algoritmos en casos normales y en casos límite, como nodos no alcanzables, pesos negativos y ciclos negativos.

.

2. Objetivos

2.1. Objetivo General

Desarrollar en C++ una aplicación que represente grafos dirigidos y ponderados mediante listas de adyacencia e implemente los algoritmos de Dijkstra, Bellman–Ford y Floyd–Warshall para calcular distancias y rutas mínimas, y que permita comparar su comportamiento en distintos escenarios.

2.2. Objetivos Específicos

- Implementar las clases nodo y grafo con las operaciones de agregar, buscar, eliminar y mostrar nodos y aristas, liberando correctamente la memoria dinámica.
- Aplicar el algoritmo de Dijkstra con cola de prioridad para obtener distancias y rutas mínimas desde un origen en grafos con pesos no negativos.
- Aplicar el algoritmo de Bellman–Ford para calcular distancias desde un origen con pesos negativos y detectar ciclos negativos.
- Aplicar el algoritmo de Floyd–Warshall para obtener la matriz de distancias mínimas entre todos los pares de nodos.
- Diseñar un menú interactivo y casos de prueba que permitan verificar los resultados y analizar las limitaciones de cada algoritmo.

3. Marco Teórico y Conceptos Clave

3.1. Grafos dirigidos y ponderados

Un grafo está formado por un conjunto de vértices o nodos V y un conjunto de aristas E. En este programa, cada nodo se identifica con un número entero y cada arista tiene un peso, que representa el costo de avanzar entre dos nodos. El grafo es dirigido: una conexión de u a v no crea automáticamente una conexión de v a u.
La representación utilizada es una lista de adyacencia. Cada objeto nodo guarda sus conexiones salientes como pares formados por un puntero al nodo de destino y el peso de la arista. La clase grafo administra los nodos y permite agregar, eliminar y mostrar conexiones.

3.2. Caminos y distancias mínimas

Un camino es una secuencia de aristas que une un origen con un destino. Su costo es la suma de los pesos recorridos. El camino mínimo es el de menor costo entre los caminos posibles. Si no existe ningún camino, el destino es no alcanzable desde el origen; el programa lo indica como «No alcanzable» o como INF en la matriz de resultados.

3.3. Algoritmos implementados

- **Dijkstra** calcula distancias y rutas mínimas desde un origen. Usa una cola de prioridad para procesar el nodo con la menor distancia conocida y registra predecesores para reconstruir las rutas. Sus resultados están garantizados cuando todas las aristas tienen pesos no negativos.
- **Bellman–Ford** calcula distancias desde un origen y admite pesos negativos. Recorre repetidamente las aristas para reducir las distancias conocidas. Una pasada adicional permite detectar un ciclo negativo alcanzable, caso en el que informa que las distancias no son confiables.
- **Floyd–Warshall** calcula las distancias entre todos los pares de nodos. Actualiza una matriz al comprobar si pasar por cada nodo intermedio reduce el costo de una ruta. Esta implementación imprime la matriz, pero no muestra una advertencia al detectar ciclos negativos.

3.4. Conceptos utilizados

Concepto	Significado en el programa
Nodo	Elemento del grafo identificado por un entero.
Arista	Conexión dirigida entre dos nodos.
Peso	Costo de recorrer una arista.
Predecesor	Nodo anterior que permite reconstruir una ruta mínima.
Ciclo negativo	Recorrido cerrado cuya suma de pesos es menor que cero.
INF	Valor usado para indicar que no se conoce un camino entre dos nodos.

4. Explicación del Código Desarrollado

4.1. Funcionamiento General

El código está organizado en tres partes. La clase nodo representa un vértice: guarda un valor entero (dato) y un vector de pares (nodo*, int) con sus aristas salientes, es decir, el puntero al nodo destino y el peso. La clase grafo guarda un vector de punteros a todos los nodos y ofrece las operaciones de construcción, visualización y los tres algoritmos. El archivo main.cpp contiene el menú interactivo que conecta al usuario con esas operaciones.
Como el grafo es dirigido, agregarArista(u, v, p) solo crea la conexión de u hacia v. Los algoritmos usan std::map para asociar el valor de cada nodo con su distancia, su estado de visita o su predecesor. Para representar «sin camino conocido» usan std::numeric_limits<int>::max() en Dijkstra y Bellman–Ford, y la constante INF = 9999999 en Floyd–Warshall.

4.2. Detalle Profundo de las Funciones

- **Clase `nodo` (`Nodo.cpp`)**

- `nodo(int d)`
  - **Función:** constructor. Inicializa el nodo con el valor `d` y una lista de adyacencia vacía.
- `int getDato() const`
  - **Función:** devuelve el valor entero que identifica al nodo.
- `void agregarArista(nodo* destino, int peso)`
  - **Función:** agrega al final del vector de adyacentes el par `{destino, peso}`, creando una arista saliente.
- `void eliminarArista(nodo* destino)`
  - **Función:** recorre los adyacentes y borra todas las aristas que apuntan a `destino`. Usa el iterador que devuelve `erase` para no invalidar el recorrido.
- `const std::vector<std::pair<nodo*, int>>& getAdyacentes() const`
  - **Función:** devuelve una referencia constante a la lista de adyacencia, para que los algoritmos la recorran sin copiarla.

Clase `grafo` (`grafo.cpp`)

- `~grafo()`
  - **Función:** destructor. Recorre el vector de nodos y ejecuta `delete` sobre cada uno para liberar la memoria dinámica.
- `void agregarNodo(int valor)`

    - **Función:** crea un nodo nuevo solo si no existe otro con el mismo valor, lo que evita duplicados.
- `void agregarArista(int origen, int destino, int peso)`
    - **Función:** busca ambos nodos y, si existen, agrega la arista dirigida desde el origen. Si alguno no existe, imprime «Origen o destino no encontrado.».
- `nodo* buscarNodo(int valor)`
    - **Función:** búsqueda lineal en el vector de nodos. Devuelve el puntero al nodo o `nullptr` si no existe. Es la base de casi todas las demás operaciones.
- `void eliminarArista(int origen, int destino)`
    - **Función:** elimina la conexión directa de origen a destino, delegando en `nodo::eliminarArista`.
- `void eliminarNodo(int valor)`
    - **Función:** elimina un nodo de forma segura. Primero borra, en todos los nodos del grafo, las aristas que apuntan a él, para no dejar punteros colgantes. Luego lo saca del vector de nodos y libera su memoria con `delete`.
- `void mostrarGrafo()`
    - **Función:** imprime cada nodo con sus adyacentes y pesos, o `[Ninguno]` si no tiene aristas salientes. Se usa para verificar las altas y bajas.
- `void Dijkstra(int valorInicial)`
    - **Función:** calcula distancias y rutas mínimas desde un origen.
    - **Funcionamiento:** inicializa todas las distancias en infinito y la del origen en 0. Usa una cola de prioridad (min-heap) para extraer siempre el nodo con menor distancia no visitada. Al visitar un nodo, relaja sus aristas: si `distancia[u] + peso < distancia[v]`, actualiza la distancia y el predecesor. Al final reconstruye cada ruta siguiendo los predecesores hacia atrás. Complejidad aproximada: O((V + E) log V). Solo es correcto con pesos no negativos.
- `void BellmanFord(int valorInicial)`
    - **Función:** calcula distancias desde un origen y admite pesos negativos.
    - **Funcionamiento:** arma una lista de todas las aristas `(u, v, peso)` y la relaja V − 1 veces. Luego hace una pasada adicional: si todavía puede mejorar alguna distancia, hay un ciclo negativo alcanzable y el programa avisa que las distancias no son confiables. Complejidad: O(V · E).
- `void FloydWarshall()`
    - **Función:** calcula la distancia mínima entre todos los pares de nodos.
    - **Funcionamiento:** convierte los valores de los nodos a índices, construye una matriz de distancias con 0 en la diagonal, `INF` donde no hay arista y el peso donde sí la hay. Con tres ciclos anidados (k, i, j) comprueba si pasar por el nodo intermedio `k` reduce el costo de `i` a `j`. Imprime la matriz con `setw` y muestra `INF` para los pares sin camino. Complejidad: O(V^3). No advierte sobre ciclos negativos; estos solo se notan por valores negativos en la diagonal.

Programa principal (`main.cpp`)

- `void mostrarMenu()`
    - **Función:** imprime las opciones agrupadas en construcción, algoritmos y pruebas.

- `int main()`
    - **Función:** crea el grafo y ejecuta un ciclo `do … while` que lee la opción, la procesa con un `switch` y termina al elegir la opción 11. Las opciones 9 y 10 reinician el grafo con `miGrafo = grafo();` y cargan los grafos de prueba 1 y 2.

4.3. Flujo de Ejecución del Sistema

1. Inicialización: main crea un objeto grafo vacío y muestra el menú.
2. Lectura de la opción: el usuario ingresa un número del 1 al 11 y el switch llama a la función correspondiente.
3. Construcción del grafo: con las opciones 1 a 4 se agregan o eliminan nodos y aristas. Con las opciones 9 y 10 se carga uno de los grafos de prueba. La opción 5 permite verificar la estructura en cualquier momento.
4. Ejecución de algoritmos: las opciones 6, 7 y 8 ejecutan Dijkstra, Bellman–Ford y Floyd–Warshall. Los dos primeros piden un nodo origen y el tercero trabaja sobre todo el grafo. Cada uno imprime sus resultados por consola.
5. Validaciones: si un nodo no existe, las funciones lo informan sin detener el programa. Si un nodo es inalcanzable, se muestra «No alcanzable» o INF.
6. Repetición y cierre: el menú se repite hasta que el usuario elige la opción 11. Al terminar main, el destructor de grafo libera todos los nodos.

5. Pruebas

Grafo con pesos positivos

Se carga el primer grafo de prueba y se ejecutan Dijkstra y Bellman–Ford desde el nodo 1, seguidos de Floyd–Warshall.

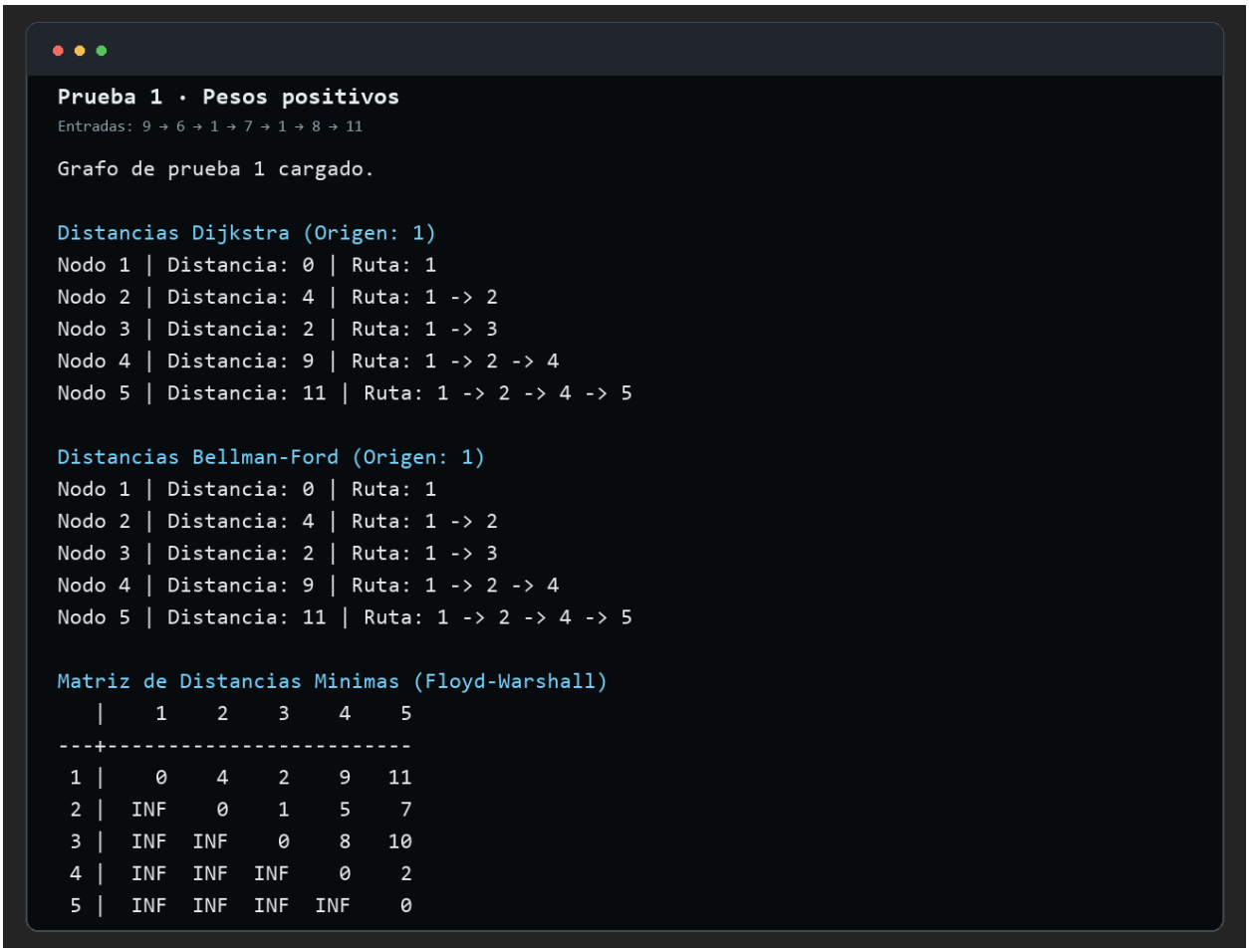


Figura 1. Distancias y rutas del grafo con pesos positivos.

Resultado. Los tres algoritmos coinciden desde el nodo 1: las distancias a los nodos 1, 2, 3, 4 y 5 son 0, 4, 2, 9 y 11, respectivamente.

5.2 Grafo con un peso negativo

Se carga el segundo grafo de prueba, que incluye la arista de 2 a 3 con peso -3. Se ejecutan Bellman–Ford desde el nodo 1 y Floyd–Warshall.

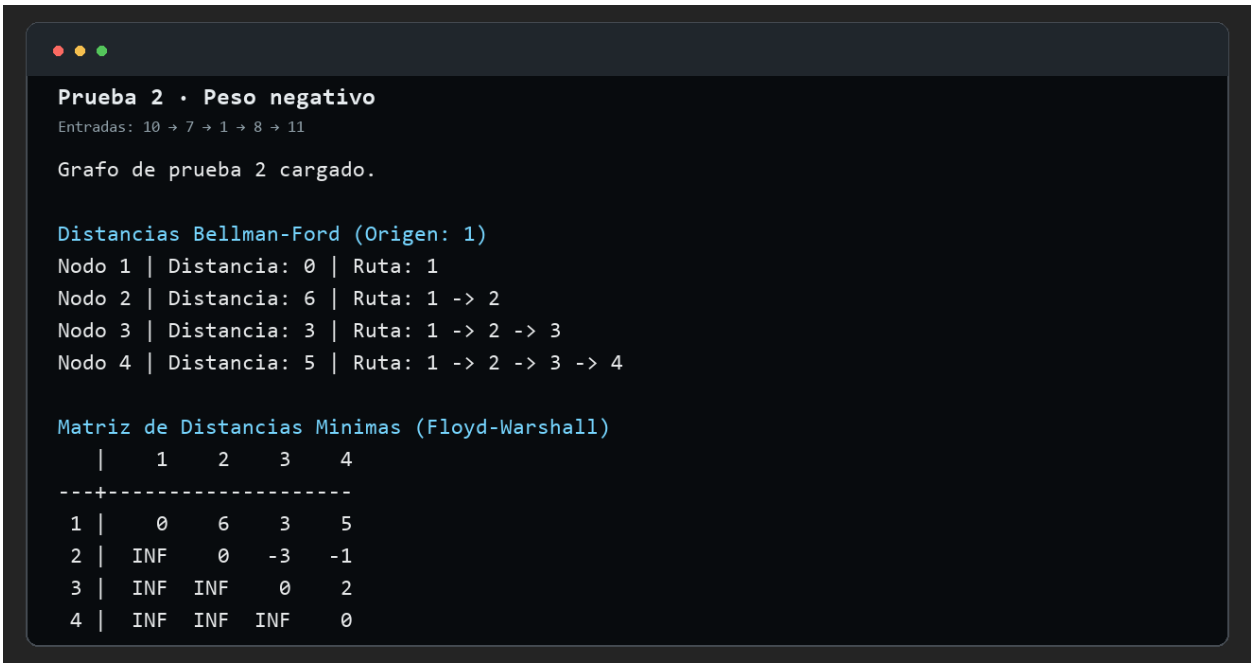


Figura 2. Distancias del grafo con una arista de peso negativo.

Resultado. Ambos algoritmos calculan distancia 3 al nodo 3 y distancia 5 al nodo 4 desde el nodo 1.

5.3 Nodo no alcanzable

Se crean los nodos 1, 2 y 3. La única arista va de 1 a 2 y tiene peso 5. Se ejecutan los tres algoritmos.

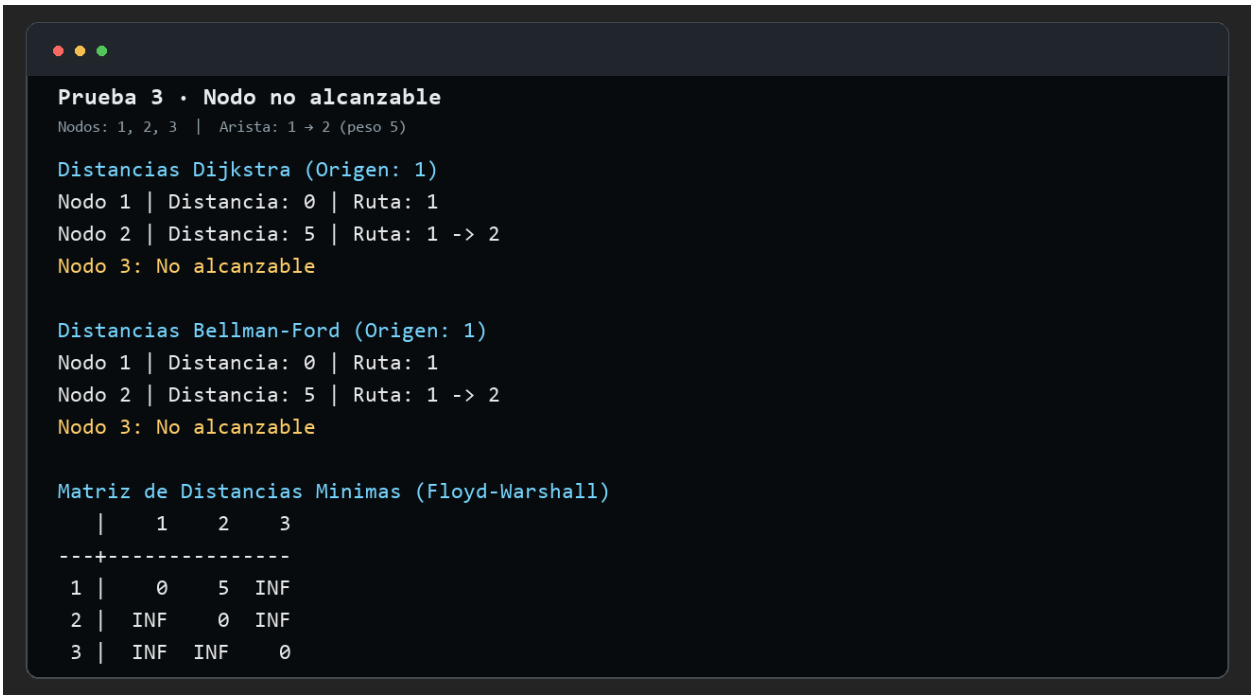


Figura 3. Identificación del nodo 3 como no alcanzable.

Resultado. Dijkstra y Bellman–Ford muestran «Nodo 3: No alcanzable» desde el origen 1. Floyd–Warshall muestra INF para el trayecto de 1 a 3.

5.4 Eliminación de un nodo

Se crean las aristas de 1 a 2 y de 3 a 2. Después de mostrar el grafo, se elimina el nodo 2 y se vuelve a mostrar la estructura.

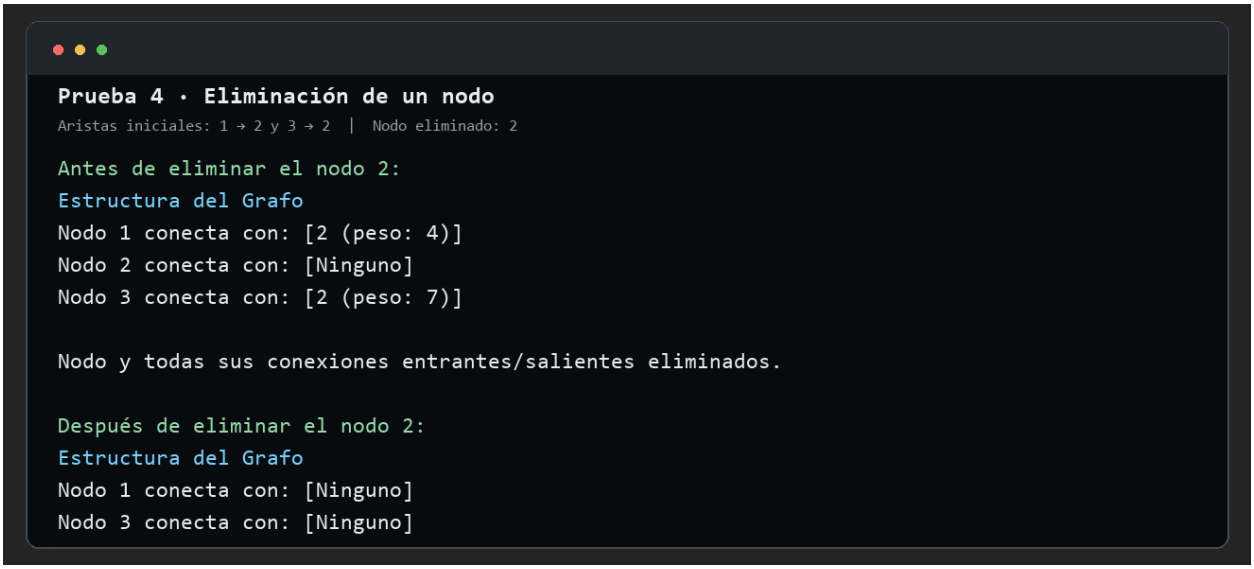


Figura 4. Estructura antes y después de eliminar el nodo 2.

Resultado. Tras la eliminación, permanecen los nodos 1 y 3, ambos sin conexiones hacia el nodo eliminado.

5.5 Detección de un ciclo negativo

Se crean dos aristas de peso -1, una de 1 a 2 y otra de 2 a 1. Su recorrido cerrado tiene peso total -2.

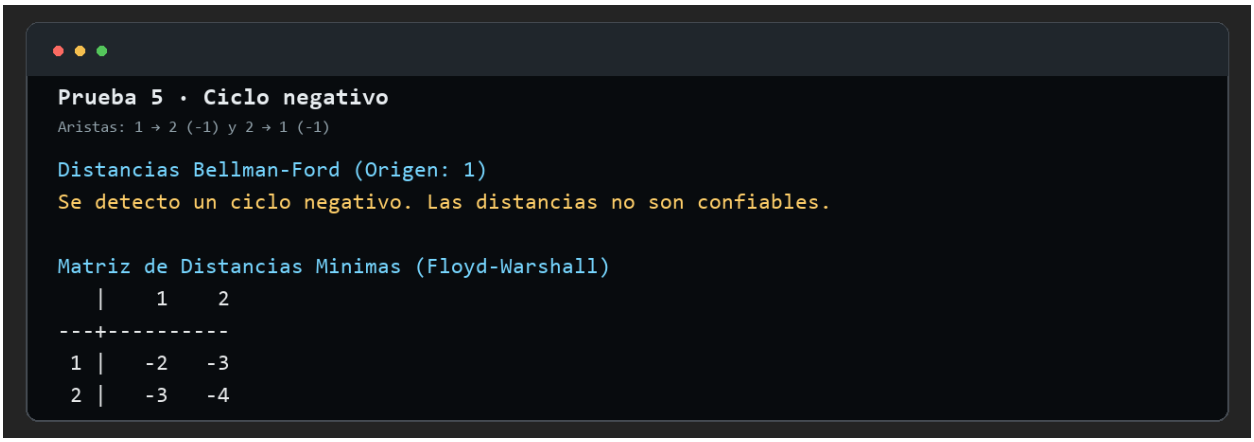


Figura 5. Aviso de Bellman–Ford y matriz de Floyd–Warshall.

Resultado. Bellman–Ford advierte que las distancias no son confiables. Floyd–Warshall imprime valores negativos en la diagonal, pero no presenta una advertencia.

5.6 Límite de Dijkstra con pesos negativos

Se construye un grafo con una arista de peso -3 y se comparan Dijkstra y Bellman–Ford desde el nodo 1.



Figura 6. Diferencia entre Dijkstra y Bellman–Ford para el nodo 4.

Resultado. Dijkstra informa distancia 2 al nodo 4, mientras que Bellman–Ford obtiene la distancia correcta, 0, por la ruta 1 → 3 → 2 → 4.

6. Conclusión

Se implementó un grafo dirigido y ponderado con listas de adyacencia, con operaciones seguras de inserción y eliminación, y se programaron tres algoritmos de caminos mínimos que se probaron en casos normales y límite.

Las pruebas confirmaron que, con pesos positivos, Dijkstra, Bellman–Ford y Floyd–Warshall entregan las mismas distancias (0, 4, 2, 9 y 11 desde el nodo 1). También mostraron que Dijkstra no es confiable con pesos negativos: en la prueba 6 calculó 2 como distancia al nodo 4, cuando la distancia correcta, obtenida por Bellman–Ford, es 0. Bellman–Ford maneja pesos negativos y detecta ciclos negativos, a cambio de mayor costo computacional, $O(V \cdot E)$. Floyd–Warshall resuelve todos los pares con $O(V^3)$, pero esta implementación no advierte la presencia de ciclos negativos.

En cuanto a la elección del algoritmo:

Dijkstra: es la mejor opción con pesos no negativos desde un solo origen.

Bellman–Ford: es necesario cuando puede haber pesos negativos.

Floyd–Warshall: conviene cuando se necesitan todas las distancias y el grafo es pequeño o denso.

Como mejoras futuras se proponen: agregar a Floyd–Warshall la detección de ciclos negativos revisando la diagonal, usar nodo* o índices en lugar de búsquedas lineales para escalar a grafos grandes, y reemplazar miGrafo = grafo(); por un método limpiar() para evitar fugas de memoria al reiniciar el grafo.